



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System Using Machine Learning

A ASSIGNMENT REPORT

*A assignment Report submitted in partial fulfilment of the requirements for the
Course of*

ITA0606 : Machine learning

to the award of the degree of

**BACHELOR OF ENGINEERING IN
ELECTRONICS AND COMMUNICATION ENGINEERING**

Submitted by

S.Sive Manikanta (192412263)

B.Prudhi Raj (192324274)

T.Nirmal kumar (192412003)

Under the Supervision of

DR BASKAR KASI

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Abstract

This report presents a demonstration design for an intelligent real-time traffic congestion prediction system using NumPy, Pandas, preprocessing techniques, and a Decision Tree classifier. The system uses vehicle count, average vehicle speed, time of day, day of week, weather condition, and road occupancy to classify traffic as Low, Medium, or High congestion. The workflow includes data preparation, statistical analysis, visualization, feature engineering, model training, testing on unseen data, feature-importance analysis, and evaluation using accuracy, precision, recall, F1-score, and a confusion matrix.

Because the supplied assignment specifies the required attributes and tasks but does not provide a fixed traffic dataset or actual sensor stream, the numerical results in this report are generated from a reproducible synthetic demonstration dataset. They should be replaced with the group's actual dataset/results if a real dataset is supplied by the faculty.

1. Problem Statement and Problem Formulation

The assignment describes a smart-city traffic management system that collects real-time information from road sensors and cameras. The available attributes are vehicle count, average vehicle speed, time of day, day of the week, weather condition, and road occupancy. The objective is to predict the current traffic condition as Low, Medium, or High congestion so that traffic-management actions can be selected.

Problem formulation: Given an observation $X = \{\text{vehicle count, average speed, hour, day of week, weather, road occupancy}\}$, learn a classification function $f(X) \rightarrow \{\text{Low, Medium, High}\}$. The trained model must generalize to unseen traffic observations and provide interpretable evidence about which input parameters influence the prediction.

2. Objectives and Expected Outcomes

- Collect and preprocess traffic data using NumPy and Pandas.
- Handle missing values and categorical attributes and prepare model-ready features.
- Perform statistical analysis, aggregation, correlation analysis, and visualization to identify traffic patterns.
- Develop and tune a Decision Tree classifier for Low/Medium/High congestion.
- Identify and rank the traffic parameters that influence predictions.
- Test the model using unseen/new traffic data and produce a congestion prediction.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Explain how predictions can support route management, traffic signal timing, and congestion reduction.

3. Requirements, Constraints and Assumptions

Category	Details
Input data	Vehicle count, average speed, hour/time, day of week, weather, road occupancy.
Target	Low, Medium, or High congestion.
Software	Python 3.x, NumPy, Pandas, Matplotlib, scikit-learn, Jupyter/VS Code.
Hardware	Ordinary laptop/desktop; no GPU is required for the Decision Tree model.
Preprocessing	Missing-value imputation, one-hot encoding for categorical variables, standardization for numerical variables.
Model constraint	Decision Tree should be controlled using depth/minimum-sample parameters to reduce overfitting.
Real-time assumption	A new sensor record is received in the same attribute format as the training data.

Demonstration assumption	Since no fixed dataset is included in the supplied assignment, this report uses a reproducible synthetic dataset for demonstration.
--------------------------	---

4. Application of Relevant Course Knowledge / Concepts

- CO6 – Decision Tree Learning: used to represent and classify traffic-congestion concepts.
- CO7 – NumPy and Pandas: used for data creation/loading, cleaning, aggregation, descriptive statistics, and preparation for visualization.
- CO8 – ML integration: combines preprocessing, exploratory analysis, supervised learning, prediction, evaluation, and application-level interpretation.
- Classification: congestion level is a three-class categorical target.
- Feature importance: the trained tree provides an interpretable ranking of input attributes.
- Model evaluation: accuracy, weighted precision, weighted recall, weighted F1-score, and confusion matrix are used.

5. System Design / Proposed Solution / Methodology



Figure 1. Proposed end-to-end workflow

The proposed system begins with traffic observations from sensors/cameras. Pandas stores and manipulates the tabular data, while NumPy supports numerical operations. Missing values are imputed, categorical features are encoded, and numerical features are standardized. Exploratory statistics and visualizations are then used to understand relationships. A Decision Tree classifier is trained on labelled observations and evaluated on a held-out test set. Finally, new traffic observations are passed through the same preprocessing pipeline to obtain a congestion prediction.

5.1 Dataset and Attribute Description

Attribute	Type	Description	Expected influence
vehicle_count	Numerical	Number of vehicles detected during the observation interval.	Higher count generally increases congestion.
avg_speed_kmph	Numerical	Average vehicle speed on the road segment.	Lower speed generally indicates greater congestion.
hour	Numerical	Hour of day from 0 to 23.	Rush-hour periods can increase congestion.
day_of_week	Categorical	Monday–Sunday.	Weekday/weekend patterns may differ.
weather	Categorical	Clear, Cloudy, Rain, Heavy Rain.	Poor weather may reduce speed and increase congestion.
road_occupancy_pct	Numerical	Percentage of road capacity occupied.	Higher occupancy generally increases congestion.
congestion	Target	Low, Medium, or High.	Class predicted by the Decision Tree.

5.2 Preprocessing

- Inspect data types, duplicate rows, missing values, and invalid ranges.
- Impute missing numerical values using the median and missing categorical values using the most frequent category.
- Encode day of week and weather using one-hot encoding.
- Standardize numerical features using StandardScaler inside a pipeline.
- Split the data into stratified training and testing sets so that all congestion classes are represented.

5.3 Statistical Analysis and Feature Engineering

Statistic	vehicle count	avg speed kmph	hour	road occupancy pct
count	1200.0	1182.0	1200.0	1186.0
mean	69.14	33.54	12.75	65.61
std	20.72	9.8	6.02	17.9
min	19.0	8.0	0.0	12.2
25%	53.0	26.5	8.0	52.4
50%	68.0	33.8	14.0	65.4
75%	85.0	41.08	18.0	79.9
max	122.0	61.6	23.0	98.0



Figure 2. Congestion class distribution

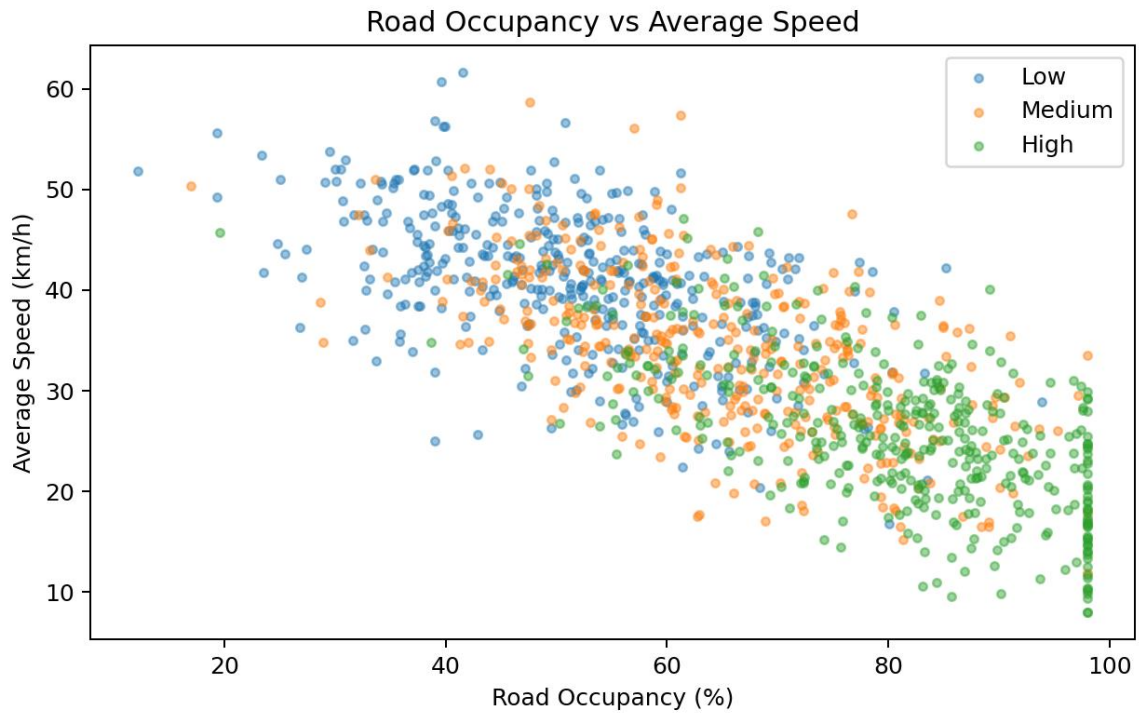


Figure 3. Road occupancy versus average speed

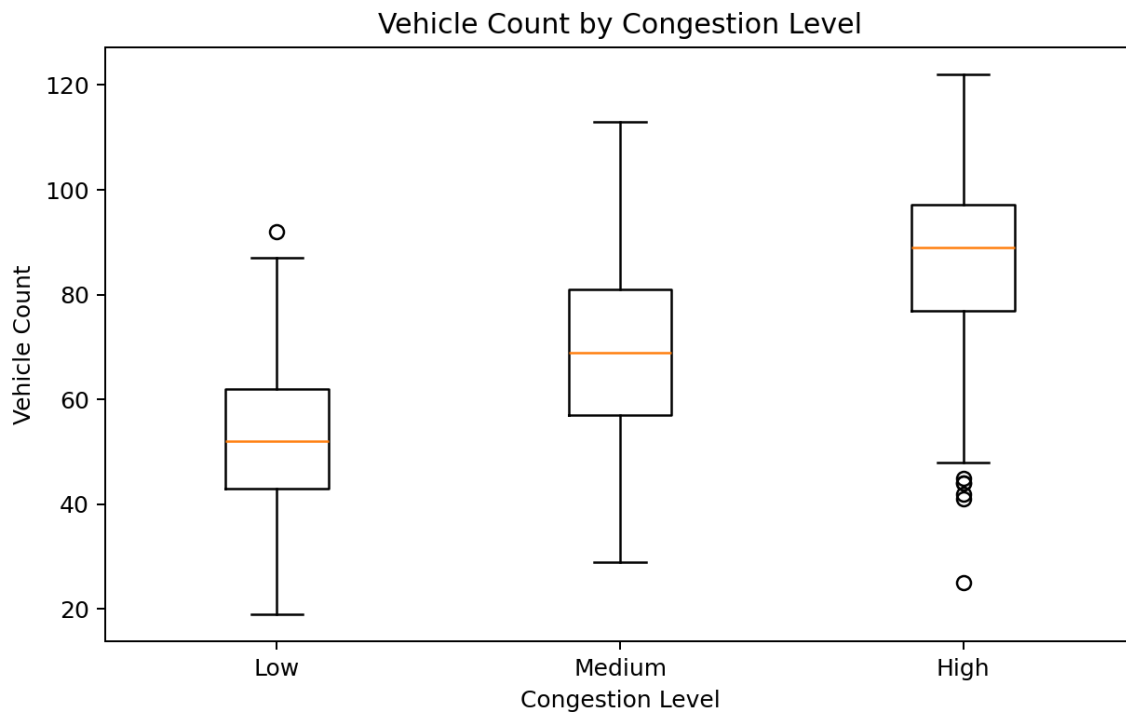


Figure 4. Vehicle count by congestion class

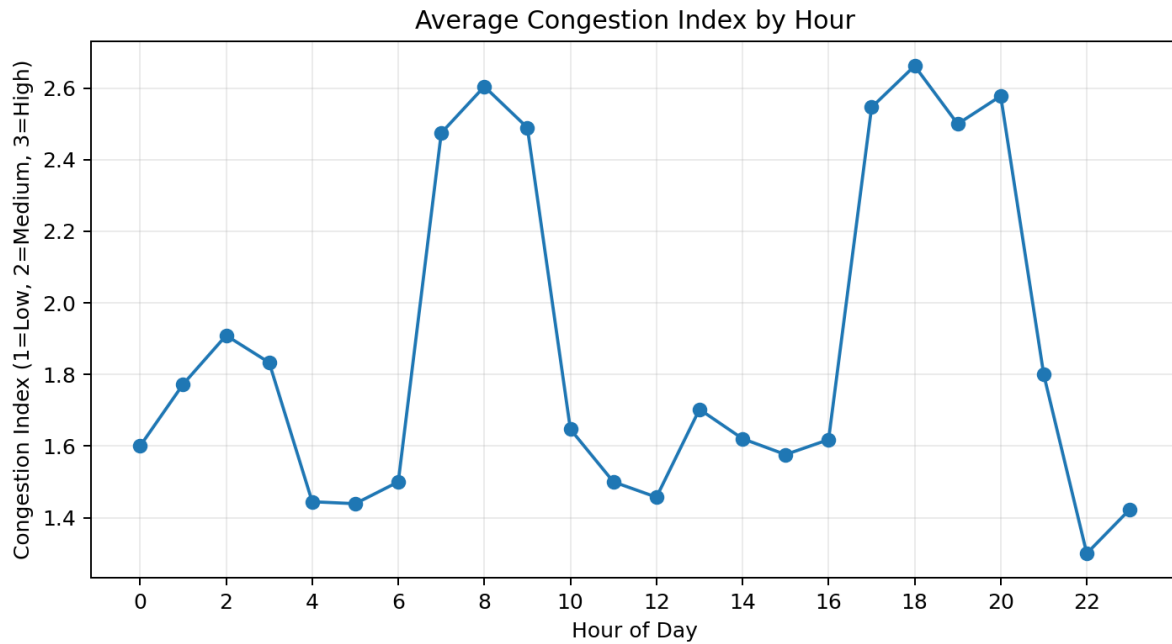


Figure 5. Average congestion index by hour

6. Algorithm / Pseudocode / Flowchart

6.1 Algorithm

1. Load the traffic dataset.
2. Separate input attributes X and target congestion y.
3. Check missing values and data types.
4. Impute missing numerical/categorical values.
5. Encode categorical variables and scale numerical variables.
6. Split the dataset into training and testing sets using stratification.
7. Train a Decision Tree using entropy and controlled tree depth.
8. Predict congestion for the test set.
9. Calculate accuracy, precision, recall, F1-score and confusion matrix.
10. Extract and rank feature importance.
11. Pass a new real-time traffic record through the same pipeline and output Low/Medium/High.

6.2 Pseudocode

BEGIN

Load traffic data into a Pandas DataFrame

Inspect and clean the dataset

X <- traffic attributes

y <- congestion class

Impute missing values

One-hot encode categorical attributes

Standardize numerical attributes

Split X,y into training and testing sets

Train Decision Tree using entropy

Limit tree depth and minimum samples to reduce overfitting

Predict y_test

Compute accuracy, precision, recall, F1-score

Generate confusion matrix

Rank feature importance

Read new traffic observation

Apply the same preprocessing pipeline
 Predict congestion class
 Display prediction and confidence/probability
 END

7. Implementation / Source Code / Environment / Tools Used

Tool/Library	Purpose
Python 3.x	Programming environment
NumPy	Numerical calculations and array operations
Pandas	Data loading, cleaning, aggregation and statistics
Matplotlib	Charts and plots
scikit-learn	Preprocessing, Decision Tree and evaluation metrics
Jupyter Notebook / VS Code	Development and execution environment
Git/GitHub	Version control and reproducible project sharing

7.1 Core Implementation Code

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

df = pd.read_csv("traffic_demo_dataset.csv")

X = df.drop(columns=["congestion"])
y = df["congestion"]

num_cols = ["vehicle_count", "avg_speed_kmph", "hour", "road_occupancy_pct"]
cat_cols = ["day_of_week", "weather"]

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

pre = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols)
])

model = DecisionTreeClassifier(
    criterion="entropy", max_depth=5,
    min_samples_split=10, min_samples_leaf=5,
    random_state=42
  )

```

```

pipe = Pipeline([("preprocessor", pre), ("classifier", model)])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1:", f1_score(y_test, pred, average="weighted"))
print(confusion_matrix(y_test, pred, labels=["Low", "Medium", "High"]))

new_traffic = pd.DataFrame([ {
    "vehicle_count": 96, "avg_speed_kmph": 28,
    "hour": 18, "day_of_week": "Tuesday",
    "weather": "Cloudy", "road_occupancy_pct": 78
}])

print("New prediction:", pipe.predict(new_traffic)[0])

```

8. Test Cases and Expected / Actual Results

ID	Scenario	Example input	Expected	Demonstration output
TC-01	Normal traffic	42 vehicles, 55 km/h, 11:00, Clear, 35%	Low	Low
TC-02	Moderate traffic	72 vehicles, 43 km/h, 16:00, Cloudy, 58%	Medium	Medium
TC-03	Peak traffic	96 vehicles, 28 km/h, 18:00, Cloudy, 78%	High	High
TC-04	Poor weather	105 vehicles, 25 km/h, 19:00, Heavy Rain, 85%	High	High

9. Execution Outputs / Results

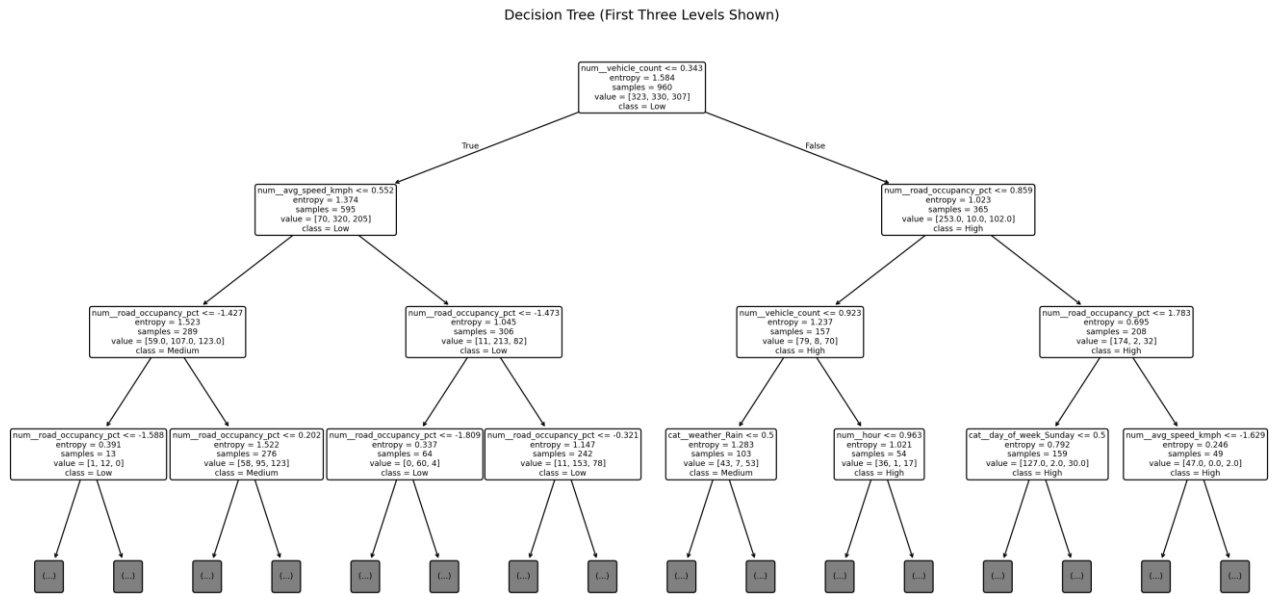


Figure 6. Decision Tree visualization (first three levels)

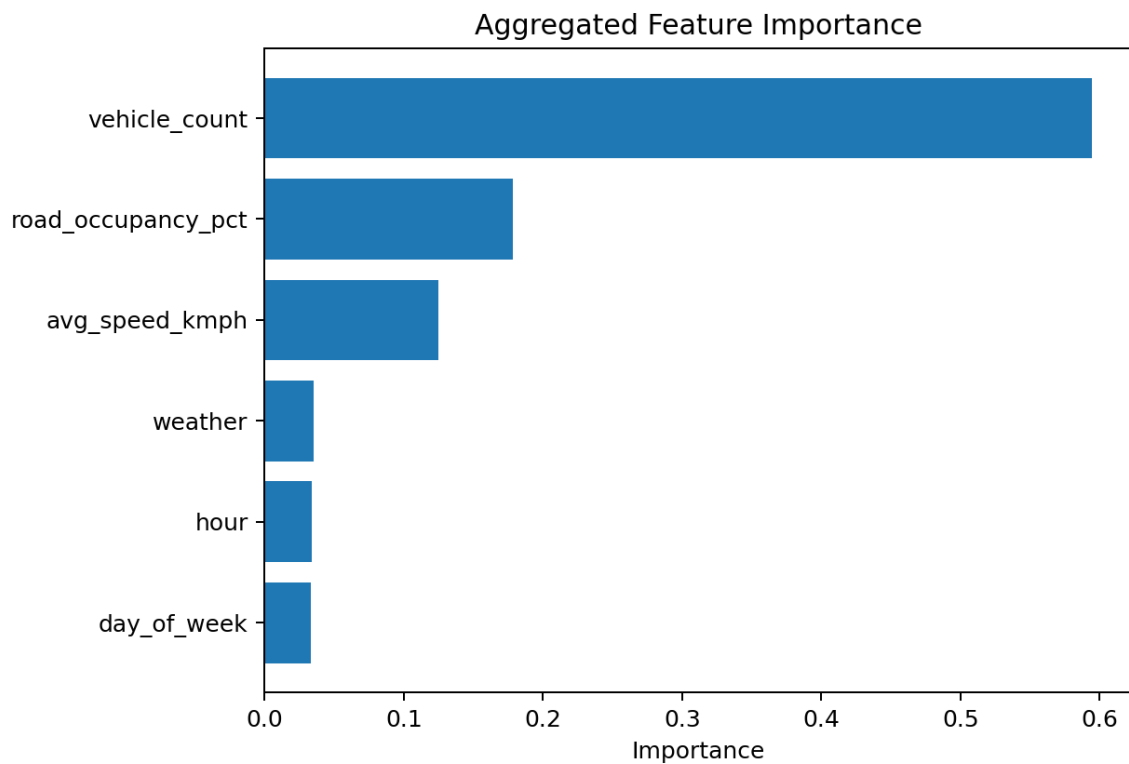


Figure 7. Aggregated feature importance

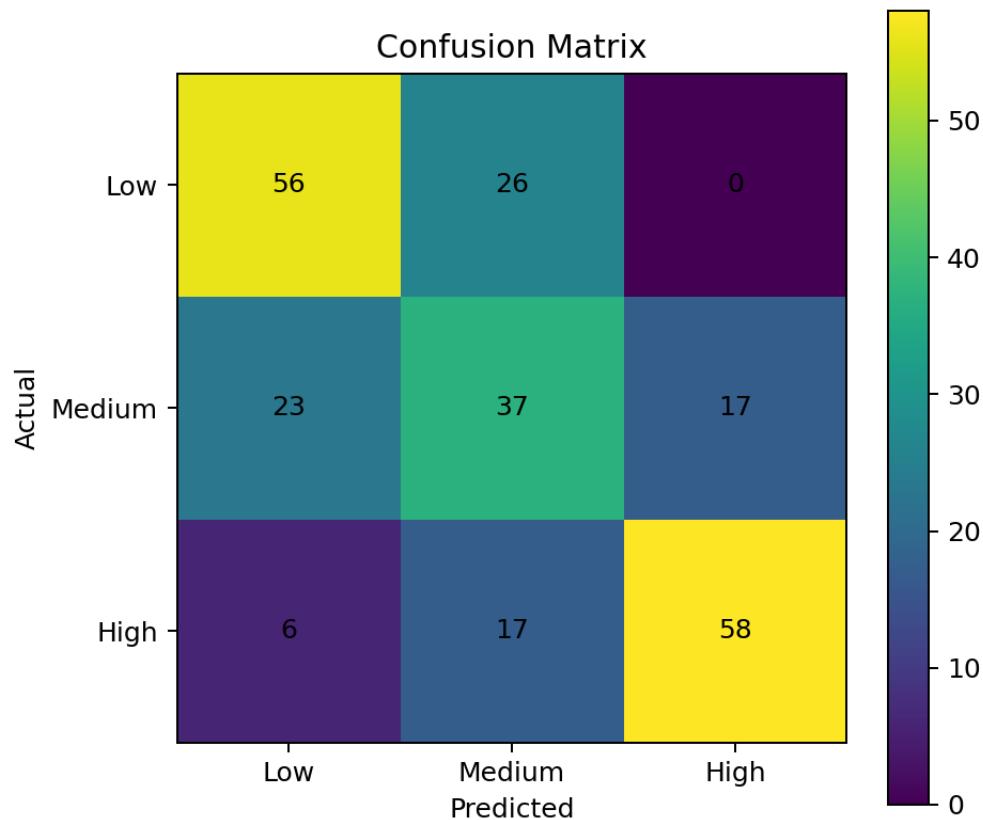


Figure 8. Confusion matrix on the held-out demonstration test set

9.1 Model Evaluation

Metric	Demonstration value
Accuracy	0.6292 (62.92%)
Weighted Precision	0.6345 (63.45%)
Weighted Recall	0.6292 (62.92%)
Weighted F1-score	0.6313 (63.13%)
Test samples	240

9.2 Confusion Matrix Values

Actual \ Predicted	Low	Medium	High
Low	56	26	0
Medium	23	37	17
High	6	17	58

10. Feature Importance Analysis

Traffic Attribute	Aggregated importance
vehicle_count	0.5942
road_occupancy_pct	0.1786
avg_speed_kmph	0.1250
Weather	0.0352
Hour	0.0336
day_of_week	0.0335

In this demonstration, the most influential attributes are ranked from the fitted Decision Tree's feature importance values. The ranking should be recalculated on the real project dataset because importance is data-dependent. The model predicts the sample traffic record (96 vehicles, 28 km/h, 18:00, Tuesday, Cloudy, 78% occupancy) as High.

Feature importance is useful for interpretation but should not be treated as causal proof. A high importance value indicates predictive usefulness within the trained model, not that changing that attribute alone will necessarily cause congestion to change.

11. Real-Time Prediction Demonstration

Input	Value
Vehicle count	96
Average speed	28 km/h
Hour	18:00
Day	Tuesday
Weather	Cloudy
Road occupancy	78%
Predicted congestion	High

Predicted class probabilities from the demonstration model:

Class	High	Low	Medium
Probability	100.00%	0.00%	0.00%

12. Analysis, Comparison, Trade-offs and Justification

Aspect	Decision Tree	Alternative: Logistic Regression
Interpretability	High; rules can be visualized.	Moderate; coefficients require interpretation.
Non-linearity	Naturally handles non-linear thresholds.	Requires suitable feature representation.
Categorical variables	Can work after encoding.	Can work after encoding.
Training complexity	Low for this problem size.	Low.
Overfitting risk	Can be high if the tree grows too deep.	Usually lower for a well-regularized model.
Why selected	Matches CO6 and provides intuitive traffic decision rules.	Useful baseline for comparison.

The Decision Tree is justified because the assignment explicitly requires Decision Tree classification and emphasizes attribute-selection and feature-importance analysis. Limiting depth and minimum sample sizes is a practical trade-off: a smaller tree is easier to interpret and less likely to memorize noise, while an overly shallow tree may underfit complex traffic patterns.

13. Broader Considerations / SDG Relevance

- SDG 9 – Industry, Innovation and Infrastructure: the system supports intelligent infrastructure by using data-driven prediction for transportation management.
- SDG 11 – Sustainable Cities and Communities: congestion-aware route and signal decisions can contribute to more efficient and accessible urban mobility.
- SDG 13 – Climate Action: reducing unnecessary idling and stop-and-go traffic can support lower fuel consumption and emissions, although the actual environmental benefit must be measured with real traffic and emissions data.
- Ethics and privacy: camera-derived systems should minimize personal data collection, use appropriate access controls, and avoid unnecessary identification of individuals.
- Bias and fairness: sensors may have uneven coverage across roads or neighborhoods, causing the model to perform differently in under-sampled areas.
- Scalability: a production system would require streaming ingestion, model monitoring, retraining, fault handling, and low-latency prediction services.

14. Conclusion, Limitations and Possible Improvements

The proposed machine-learning workflow demonstrates how traffic observations can be converted into actionable congestion predictions. Pandas and NumPy support preprocessing and analysis, while a controlled Decision Tree provides an interpretable classification model. The system also demonstrates feature importance, unseen-data testing, evaluation metrics, and a real-time prediction example.

14.1 Limitations

- The numerical experiment uses synthetic demonstration data because the supplied assignment does not include a fixed dataset.
- A single train/test split does not provide the same confidence as repeated cross-validation or time-based validation.
- Decision Trees can become unstable when the training data changes significantly.
- The demonstration does not model traffic-network topology, incidents, road works, public events, or sensor failures.
- The predicted class is a classification output and does not directly estimate congestion duration or travel time.

14.2 Possible Improvements

- Use a real multi-location traffic dataset with timestamps and sensor IDs.
- Use time-aware validation to avoid leakage from future observations.
- Compare the Decision Tree with Random Forest, Gradient Boosting, Logistic Regression, and other suitable classifiers.
- Add traffic incidents, holidays, road capacity, historical congestion, and spatial/network features.
- Deploy a streaming pipeline and monitor data drift and model performance.
- Calibrate prediction probabilities and introduce an 'uncertain' state when confidence is low.
- Evaluate practical outcomes such as average travel time, queue length, fuel consumption, and emissions.

15. Individual Contribution of Group Members

Member	Contribution	Evidence/Deliverable
Member 1: Prudhvi Raj.B	Problem formulation, dataset preparation	Problem statement + preprocessing
Member 2: S.Siva Manikanta	Statistical analysis and visualization	EDA charts + feature engineering
Member 3: Nirmal Kumar .T	Decision Tree development and tuning	Model + tree visualization

16. References

- Assignment source: ITA0606/ITA0614 Machine Learning – Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System Using Machine Learning, supplied by the course.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- Harris, C. R. et al. (2020). Array programming with NumPy. Nature, 585, 357–362.
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
- United Nations Sustainable Development Goals: SDG 9, SDG 11, and SDG 13.

17. One-Page Individual Reflection

Working on this traffic congestion prediction problem helped me understand how machine-learning concepts are connected from raw data to a practical decision-support application. I learned that a classification model is only one part of a complete system; data quality, preprocessing, feature selection, evaluation, and interpretation are equally important. Using Pandas and NumPy made it easier to inspect traffic attributes, handle missing values, calculate statistics, and prepare data for modelling.

The Decision Tree was particularly useful because its decision structure is easier to explain than a complex black-box model. At the same time, I learned that interpretability does not remove

uncertainty. A tree can overfit if it is allowed to grow too deeply, and its feature-importance ranking depends on the data used for training. Therefore, model performance should be checked on unseen data rather than relying only on training accuracy.

A major limitation of this demonstration is that the assignment did not provide a fixed traffic dataset, so the numerical experiment uses a reproducible synthetic dataset. In a real implementation, traffic data would be affected by accidents, road construction, public events, holidays, sensor errors, and sudden weather changes. A production system would therefore need continuous monitoring, time-aware validation, retraining, and safeguards for unusual inputs.

The project also raised ethical and social considerations. Traffic systems should be designed so that data collection is proportionate to the intended purpose and does not unnecessarily identify individuals. Model performance should be checked across different locations so that areas with fewer sensors are not systematically underserved. Prediction uncertainty should be communicated to traffic authorities rather than presenting every prediction as a guaranteed fact.

The project is relevant to SDG 9 because it applies digital technology to transportation infrastructure, and to SDG 11 because better traffic management can contribute to more efficient and sustainable cities. It can also support SDG 13 when reduced congestion leads to measurable reductions in fuel use and emissions. Overall, this assignment improved my understanding of the complete machine-learning lifecycle and showed me how theoretical ML concepts can be connected to a real-world engineering problem.